

DevReview

Real-Time Community Code Review & Peer Feedback Hub

Project Type: Full Stack Web Application **Architecture:** MERN + WebSockets + Docker **Target Alignment:** 10 Lab Experiments

1. Executive Summary & Core Vision

DevReview is a developer-centric social feed platform—combining the engagement model of **Reddit/daily.dev** with the precision code-review mechanics of **GitHub Pull Requests**. Instead of static text links or generic discussion threads, every post on DevReview centers around a code snippet, architectural pattern, or debugging dilemma.

Developers post code requests tagged by topic (e.g., `#react`, `#express`, `#security`) asking for community feedback, refactor suggestions, or performance audits. Reviewers can line-comment directly on syntax-highlighted code, upvote high-quality contributions, submit refactor diffs, and chat in real-time via WebSockets. The platform gamifies peer mentorship through developer reputation (Karma) and verified reviewer badges.

Key Value Proposition: Converts traditional, dry peer-code reviews into a collaborative, gamified, public learning feed where developers build reputations while learning best practices.

2. Problem Statement & Solution

The Problem

- Standard Git platforms (GitHub/GitLab) treat reviews as strict, async gatekeepers tied to entire repos.
- Junior developers and students lack lightweight platforms to ask targeted questions (e.g., *"Is this async loop clean?"*).
- Forums like StackOverflow heavily penalize opinionated code discussions or refactoring advice.

The Solution

- An open, card-based social feed dedicated solely to code review and peer feedback.
- Inline, line-by-line commenting on syntax-highlighted code snippets.
- Real-time WebSocket interaction (live upvote counts, active reader presence, real-time comment streams).
- Community gamification with upvotes, solution acceptance, and user Karma scores.

3. Key Feature Modules

3.1 Community Feed & Discovery

- **Algorithmic Sorting:** Filter feed by *Hot*, *Top Upvoted*, *Unanswered/Needs Review*, and *Latest*.
- **Topic Tags & Search:** Tag posts with technologies (`#mongodb` , `#react` , `#tailwind`) and filter instantly.
- **Interactive Code Cards:** Feed cards preview syntax-highlighted snippets, upvote counters, tag badges, and reviewer stats.

3.2 Interactive Line-by-Line Code Viewer

- **Syntax Highlighting:** Embedded Monaco Editor / PrismJS code viewer with line numbers.
- **Line Comments:** Click any line number to attach a targeted feedback thread or question.
- **Refactor Submissions:** Reviewers can submit side-by-side proposed code fixes (diffs).

3.3 Real-Time WebSocket Layer

- **Live Upvote & Counter Sync:** Upvotes and comment counts update in real time across all connected browsers without page reloads.
- **Presence Indicators:** Shows live viewer count on active code posts (e.g., *"4 developers inspecting this code"*).
- **Instant Notifications:** Authors receive live toast notifications when someone line-comments or accepts a refactor.

3.4 Role-Based Access Control (RBAC) & Gamification

- **User Roles:** Student / Developer (Post, comment, vote) vs. Moderator / Admin (Pin top reviews, flag bad posts, grant badges).
- **Karma & Reputation:** Points awarded when comments are upvoted or marked as "Accepted Refactor".

4. System Architecture & Stack

DevReview follows a decoupled, containerized MERN architecture augmented with WebSockets for real-time bi-directional streaming.

Layer	Technologies Used	Responsibility
Frontend	React 18, Tailwind CSS, Monaco Editor / PrismJS, Lucide Icons	Responsive UI, state management, interactive code viewer, real-time socket events.
State Management	Redux Toolkit / Context API, React Hooks	Global auth state, socket instance, active feed filters, notification drawer.
Backend API	Node.js, Express.js, JWT, Express-Validator	RESTful endpoints, request validation, authentication, business logic, RBAC.
Real-Time Engine	Socket.io	Bi-directional event channels for live upvotes, presence, and instant chat/comments.
Database	MongoDB, Mongoose ODM	Document storage for Users, Posts, Line Comments, Refactors, and Tags.
DevOps & CI/CD	Docker, Docker Compose, GitHub Actions, Vercel/Render	Multi-container orchestration, automated linting/testing, automatic cloud deployment.

5. MongoDB / Mongoose Schema Design

The database schema is optimized for nested comment threads and line-referenced code feedback.

```
// 1. User Schema
const UserSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }, // Hashed with bcrypt
  role: { type: String, enum: ['User', 'Moderator', 'Admin'], default: 'User' },
  karma: { type: Number, default: 0 },
  avatar: { type: String, default: '' },
}, { timestamps: true });
```

```
// 2. Code Post Schema
const PostSchema = new mongoose.Schema({
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true },
  description: { type: String },
  codeSnippet: { type: String, required: true },
  language: { type: String, required: true }, // e.g. 'javascript', 'python'
  tags: [{ type: String }], // e.g. ['react', 'performance']
  upvotes: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  upvoteCount: { type: Number, default: 0 },
  status: { type: String, enum: ['Open', 'Resolved'], default: 'Open' },
}, { timestamps: true });

// 3. Line Comment & Refactor Schema
const CommentSchema = new mongoose.Schema({
  post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post', required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  lineNumber: { type: Number, default: null }, // Null for general comments, number for line comments
  content: { type: String, required: true },
  proposedCode: { type: String }, // Optional refactor code diff
  isAccepted: { type: Boolean, default: false },
  upvotes: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
}, { timestamps: true });
```

6. REST API Endpoint Specification

Method	Endpoint	Auth Level	Description
POST	/api/auth/register	Public	Register a new user account with hashed password.
POST	/api/auth/login	Public	Authenticate user and return signed JWT token.
GET	/api/posts	Public	Fetch paginated code posts with sorting (<i>hot</i> , <i>top</i> , <i>latest</i>) & tag filters.
POST	/api/posts	User (JWT)	Create a new code review request with snippet & tags.
PUT	/api/posts/:id/upvote	User (JWT)	Toggle upvote on a post; recalculates Karma & triggers Socket event.
GET	/api/posts/:id/comments	Public	Get all line-by-line and general comments for a post.
POST	/api/posts/:id/comments	User (JWT)	Add a comment or refactor attached to a specific line number.
DELETE	/api/posts/:id	Admin / Mod	Delete offensive or spam posts (RBAC Protected).

7. Real-Time WebSockets Event Protocol

Event Name	Direction	Payload	Action Taken
join_post_room	Client → Server	{ postId }	Subscribes client socket to live updates for a specific post.
post_upvoted	Server → Room	{ postId, newUpvoteCount }	Updates upvote pill counter seamlessly across all connected viewers.

Event Name	Direction	Payload	Action Taken
new_line_comment	Server → Room	{ commentData, lineNumber }	Injects new comment indicator directly next to the code line.
presence_update	Server → Room	{ activeViewersCount }	Displays live count of active developers reading the post.

8. Direct Alignment with 10 Lab Experiments

The table below provides an explicit mapping showing how DevReview fulfills every requirement of your syllabus:

Exp #	Lab Experiment Topic	Project Milestone Implementation in DevReview
1	Tailwind CSS UI	Responsive dark-themed Reddit layout: left tag navigation bar, central code feed, and right trending/karma sidebar.
2	React Hooks	Custom hooks (useFetchFeed , useSocket , useUpvote), controlled code input forms, and event handlers.
3	Redux / Context API	Global state managing JWT user session, active notification feed, socket connection status, and active feed filters.
4	REST API + MongoDB	Complete Mongoose CRUD architecture for Users, Code Posts, Line Comments, and Tags with relationships.
5	Secure REST APIs	Express-validator middleware sanitizing code strings, centralized error handling, rate limiting, and CORS setup.
6	JWT Authentication	JWT issue/verify middleware, protected routes, and Role-Based Access Control (RBAC: User vs Admin/Moderator).
7	Deploy using Docker	Individual production Dockerfiles for React frontend client and Node/Express backend service.
8	WebSockets	Socket.io integration for live upvotes, presence detection, real-time comment streams, and instant toast alerts.
9	Containerization	Multi-container orchestration via docker-compose.yml running Frontend, Backend, and local MongoDB container.
10	CI/CD Pipeline	GitHub Actions workflow for automated testing and continuous deployment to Vercel/Render on repository push.

9. DevOps & Deployment Blueprint

Docker Compose File (docker-compose.yml)

```
version: '3.8'
services:
  mongodb:
    image: mongo:latest
    ports: ["27017:27017"]
    volumes: [mongo-data:/data/db]
  backend:
    build: ./backend
```

```

  ports: ["5000:5000"]
  environment:
    - MONGO_URI=mongodb://mongodb:27017/devreview
    - JWT_SECRET=supersecretkey
  depends_on: [mongodb]
  |
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    depends_on: [backend]
  |
volumes:
  mongo-data:

```

GitHub Actions CI/CD (.github/workflows/deploy.yml)

```

name: CI/CD Pipeline
on:
  push:
    branches: [ main ]
  |
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Install & Test Backend
        run: |
          cd backend
          npm install
          npm test
      - name: Deploy to Production
        if: success()
        run: echo "Triggering Render/Vercel Webhooks..."

```

10. Evaluator Demo Strategy (How to Score Maximum Marks)

- Side-by-Side Browser Demo:** Open Window A (Logged in as Normal User) and Window B (Logged in as Admin). Post code in Window A, upvote in Window B, and demonstrate real-time WebSocket socket updates without refreshing either window.
- Line-by-Line Commenting Showcase:** Click on Line 14 of a code snippet, type a suggestion, and show how the comment attaches specifically to that code line.
- Role Control Demonstration:** Show that regular users cannot access administrative controls (e.g., pinning posts or deleting content), while the Admin JWT grants access to moderation tools.
- Docker Single-Command Spin-up:** Demonstrate running `docker-compose up` in terminal to launch the whole full-stack system seamlessly.